

INFORME TÉCNICO

Desarrollo de un Sistema Cognitivo usando MCP y Bases de Datos

Curso:	Computación Cognitiva para Big Data
Tema:	Conexión a herramientas externas mediante Model Context Protocol
Actividad:	Guía de Actividad 6
Tecnologías:	Python 3.10 · SQLite3 · FastMCP

1. Introducción

Las organizaciones modernas requieren sistemas capaces de consultar, procesar y analizar información almacenada en bases de datos, servicios web y plataformas externas. En este contexto, los sistemas cognitivos apoyados en modelos de lenguaje extienden sus capacidades mediante herramientas que les permiten interactuar con datos reales en tiempo de ejecución.

El **Model Context Protocol (MCP)** es un estándar abierto que define la manera en que un modelo de lenguaje se conecta con herramientas externas: bases de datos, APIs, sistemas de archivos y servicios computacionales. Esta actividad propone el desarrollo completo de un servidor MCP en Python que gestiona el inventario de una organización mediante operaciones CRUD y consultas analíticas sobre una base de datos SQLite.

2. Objetivo

Diseñar e implementar un servidor MCP en Python que permita realizar operaciones CRUD y consultas analíticas sobre una base de datos de inventario, comprendiendo cómo un sistema cognitivo puede interactuar con herramientas externas para obtener y transformar información.

3. Tecnologías utilizadas

Tecnología	Versión	Función en el proyecto
Python	3.10+	Lenguaje principal de implementación
SQLite3	Nativa	Motor de base de datos relacional embebido
FastMCP	Última	Framework para crear servidores MCP en Python
MCP Protocol	1.x	Estándar de comunicación entre LLM y herramientas

4. Arquitectura del sistema

El sistema sigue una arquitectura de tres capas:

- **Capa de datos:** Base de datos SQLite gestionada en *database.py*, con la tabla *productos* que almacena id, nombre, categoría, cantidad y precio.
- **Capa de herramientas MCP:** Servidor implementado en *server.py* mediante FastMCP. Expone nueve herramientas (tools) que el modelo de lenguaje puede invocar.
- **Capa de interfaz cognitiva:** El modelo de lenguaje (cliente MCP) recibe las definiciones de herramientas y las invoca de forma autónoma según el contexto de la conversación.

4.1 Estructura de archivos

Archivo	Descripción
database.py	Inicializa la base de datos y define DB_NAME
server.py	Servidor FastMCP con las 9 herramientas expuestas
requirements.txt	Dependencia principal: fastmcp
README.md	Instrucciones de instalación y ejecución
inventory.db	Base de datos SQLite (generada automáticamente)

5. Implementación

5.1 Base de datos (database.py)

Se crea la tabla *productos* con los campos:

```
CREATE TABLE IF NOT EXISTS productos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL,  
  categoria TEXT,  
  cantidad INTEGER NOT NULL,  
  precio REAL NOT NULL  
)
```

5.2 Herramientas CRUD (server.py)

Herramienta	Tipo	Descripción
crear_producto	CREATE	Inserta un producto con nombre, categoria, cantidad y precio
consultar_producto	READ	Retorna los datos de un producto por su id
actualizar_producto	UPDATE	Modifica la cantidad disponible de un producto
eliminar_producto	DELETE	Elimina un producto del inventario por su id
listar_productos	READ	Retorna la lista completa de productos registrados

5.3 Herramientas analíticas

Herramienta	Descripción
calcular_valor_total_inventario	Suma cantidad x precio para todos los productos
productos_agotados	Lista los productos cuya cantidad es igual a cero
producto_mas_costoso	Retorna el producto con el precio unitario mas alto
estadisticas_inventario	Total de productos, promedios de cantidad y precio, valor total

6. Pruebas funcionales

Se ejecutaron las nueve pruebas requeridas por la guía. A continuación se describe cada prueba y el resultado obtenido:

1. Crear cinco productos

Procedimiento: Se insertaron cinco productos: Laptop Dell XPS 15 (Electronica, \$4,599,000), Mouse Inalambrico (Accesorios, \$89,900), Monitor LG 27 pulgadas (Electronica, \$1,250,000), Teclado Mecanico (Accesorios, \$349,000) y Silla Ergonomica (Muebles, \$2,100,000).

Resultado: EXITOSO — IDs asignados del 1 al 5.

2. Consultar producto por id

Procedimiento: Se invoco consultar_producto(id=2).

Resultado: EXITOSO — Retorno: {id:2, nombre:"Mouse Inalambrico", categoria:"Accesorios", cantidad:45, precio:89900.0}

3. Actualizar cantidad de un producto

Procedimiento: Se invoco actualizar_producto(id=4, cantidad=20) sobre el Teclado Mecanico.

Resultado: EXITOSO — Mensaje: Producto id=4 actualizado correctamente. Nueva cantidad: 20.

4. Eliminar un producto

Procedimiento: Se creo un producto temporal (id=6) y se elimino con eliminar_producto(id=6). Se verifico que consultar_producto(6) retorna error.

Resultado: EXITOSO — Producto eliminado; consulta posterior retorna error esperado.

5. Listar todos los productos

Procedimiento: Se invoco listar_productos() y se obtuvieron los cinco productos con todos sus campos.

Resultado: EXITOSO — Lista completa retornada correctamente.

6. Calcular valor total del inventario

Procedimiento: Se invoco calcular_valor_total_inventario().

Resultado: EXITOSO — Valor total: \$86,713,500 COP.

7. Consultar productos agotados

Procedimiento: Se establecio cantidad=0 para el Teclado Mecanico (id=4) y se invoco productos_agotados().

Resultado: EXITOSO — Retorno: [{id:4, nombre:'Teclado Mecanico', cantidad:0}].

8. Identificar producto mas costoso

Procedimiento: Se invoco producto_mas_costoso().

Resultado: EXITOSO — Resultado: Laptop Dell XPS 15 con precio \$4,599,000.

9. Estadisticas generales

Procedimiento: Se invoco estadisticas_inventario().

Resultado: EXITOSO — Total: 5, prom. cantidad: 14.0, prom. precio: \$1,677,580, valor total: \$79,733,500.

7. Preguntas de analisis

1. Que papel cumple MCP en la construccion de sistemas cognitivos?

El Model Context Protocol actua como puente estandarizado entre un modelo de lenguaje y el mundo externo. Sin MCP, un LLM solo puede procesar texto dentro de su ventana de contexto. Con MCP, el modelo puede invocar herramientas (funciones Python, APIs, consultas SQL, etc.) de forma estructurada, recibir sus resultados e integrarlos en su razonamiento. Esto transforma al LLM de un sistema de generacion de texto en un agente cognitivo capaz de actuar sobre datos reales.

2. Por que es importante conectar un modelo de lenguaje con herramientas externas?

Los modelos de lenguaje tienen un conocimiento estatico, limitado a su fecha de corte de entrenamiento. La conexion con herramientas externas les permite: (a) acceder a datos actualizados en tiempo real, (b) realizar operaciones que superen su capacidad computacional intrinseca (calculos numericos precisos, consultas sobre millones de registros), (c) persistir informacion mas alla de la conversacion y (d) interactuar con sistemas empresariales reales como ERP, CRM o plataformas de e-commerce.

3. Que ventajas ofrece el uso de una base de datos frente al manejo de archivos planos?

Las bases de datos relacionales ofrecen: (a) integridad referencial y restricciones de tipo de dato, (b) lenguaje SQL para consultas complejas con filtros, agrupaciones y joins sin necesidad de cargar todos los datos en memoria, (c) transacciones ACID que garantizan consistencia ante fallos, (d) indices que aceleran las busquedas en grandes volúmenes y (e) control concurrente de accesos multiples. Los archivos planos (CSV, JSON) carecen de estas garantias y se vuelven inmanejables a medida que crece el volumen de datos.

4. Como podria ampliarse esta solucion para un escenario de Big Data?

Para escalar a Big Data se podrian adoptar: (a) motores de base de datos distribuidos como PostgreSQL + Citus o Apache Cassandra para manejar terabytes de datos, (b) Apache Kafka para ingestiones en tiempo real de movimientos de inventario, (c) Apache Spark para procesamiento analitico distribuido sobre el historial de transacciones, (d) un data lake en S3 o GCS para almacenamiento economico de datos historicos, y (e) un orquestador como Apache Airflow para pipelines de ETL automatizados. El servidor MCP actuaria como capa de abstraccion sobre estas infraestructuras, exponiendo herramientas de alto nivel al modelo de lenguaje.

5. Que riesgos se deben considerar al permitir que un modelo de IA consulte o modifique datos?

Los principales riesgos son: (a) **Inyeccion de prompts maliciosos** que manipulen al modelo para ejecutar operaciones no autorizadas (DROP TABLE, exfiltracion de datos sensibles); (b) **Falta de**

autorizacion granular: el modelo podria ejecutar herramientas fuera del alcance del usuario autenticado; (c) **Alucinaciones que generan datos incorrectos** insertados en la base de datos; (d) **Ausencia de auditoria:** sin logs de las operaciones ejecutadas, es imposible rastrear cambios no intencionales; (e) **Denegacion de servicio** por consultas mal construidas que saturan la base de datos. Se recomienda implementar autenticacion OAuth 2.0, principio de minimo privilegio, validacion de entradas y registro de auditorias completo.

8. Conclusiones

- MCP permite a los modelos de lenguaje convertirse en agentes capaces de interactuar con sistemas reales, superando las limitaciones del conocimiento estatico.
- La separacion entre la capa de datos (database.py) y la capa de herramientas (server.py) favorece la mantenibilidad y la escalabilidad del sistema.
- SQLite resulta ideal para prototipos y sistemas de baja concurrencia, pero debe reemplazarse por motores mas robustos en produccion.
- La validacion de entradas en cada herramienta MCP es esencial para prevenir operaciones incorrectas o maliciosas generadas por el modelo.
- Este patron de diseno (LLM + MCP + BD) es la base de arquitecturas de agentes cognitivos aplicables a inventarios, CRMs, analisis financiero y muchos otros dominios.